



Sravya Madipalli ✓

SQL Interview Questions E-Commerce

Schema Details

To work through the following interview questions, assume you are working with a basic e-commerce database. Below are the core tables and their structures:

Customers: Stores customer information.

- `customer\_id` (Primary Key, INT)
- `first\_name` (VARCHAR)
- `last\_name` (VARCHAR)
- `email` (VARCHAR)
- `signup\_date` (DATE)

Orders: Tracks orders placed by customers.

- `order\_id` (Primary Key, INT)
- `customer\_id` (Foreign Key, INT)
- `order\_date` (DATE)
- `total\_amount` (DECIMAL)

Schema Details

Products: Stores product information

- ``product_id`` (Primary Key, INT)
- ``product_name`` (VARCHAR)
- ``price`` (DECIMAL)
- ``category`` (VARCHAR)

order_items: Stores information about each product included in an order

- ``order_item_id`` (Primary Key, INT)
- ``order_id`` (Foreign Key, INT)
- ``product_id`` (Foreign Key, INT)
- ``quantity`` (INT)
- ``subtotal`` (DECIMAL)

Question 1: Aggregation with Group By

Scenario: Counting the total sales (in dollars) for each product category in January 2024

SOLUTION

```
SELECT p.category,  
       SUM(oi.subtotal) AS total_sales  
FROM order_items oi  
JOIN products p ON oi.product_id = p.product_id  
JOIN orders o ON oi.order_id = o.order_id  
WHERE o.order_date BETWEEN '2024-01-01' AND '2024-01-31'  
GROUP BY p.category;
```

Explanation: This query calculates the total sales for each product category by summing the subtotals of products sold in January 2024. The `JOIN` operations link the product and order details, and the `GROUP BY` function ensures the sales are aggregated by category.

Question 2: Joining Tables and Aggregation

Scenario: Retrieving the full name and total amount spent by each customer who made purchases in 2023.

SOLUTION

```
SELECT c.first_name,  
       c.last_name,  
       SUM(o.total_amount) AS total_spent  
FROM customers c  
JOIN orders o ON c.customer_id = o.customer_id  
WHERE o.order_date BETWEEN '2023-01-01' AND '2023-12-31'  
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Explanation: This query retrieves customer names and calculates the total amount spent by each customer during 2023. It uses a `JOIN` to link customer data with their respective orders, applying a `GROUP BY` to aggregate their spending.

Question 3: Window Functions with Running Totals

Scenario: Calculating the running total of sales for each day in January 2024.

SOLUTION

```
SELECT o.order_date,  
       SUM(o.total_amount) OVER (ORDER BY o.order_date)  
AS running_total  
FROM   orders o  
WHERE  o.order_date BETWEEN '2024-01-01' AND '2024-01-31';
```

Explanation: This query uses the `OVER()` clause with the `SUM()` function to calculate a running total of sales by date. It sums up sales amounts in a cumulative manner, ordered by the `order\_date`.

Question 4: Using CTEs for Complex Aggregation

Scenario: Calculating total sales per product category, but only for products that have sold more than 100 units in 2023.

SOLUTION

```
WITH product_sales AS (  
    SELECT oi.product_id,  
           SUM(oi.quantity) AS total_quantity_sold  
    FROM order_items oi  
    JOIN orders o ON oi.order_id = o.order_id  
    WHERE o.order_date BETWEEN '2023-01-01' AND '2023-12-31'  
    GROUP BY oi.product_id  
)  
SELECT p.category,  
       SUM(oi.subtotal) AS total_sales  
FROM product_sales ps  
JOIN products p ON ps.product_id = p.product_id  
JOIN order_items oi ON ps.product_id = oi.product_id  
WHERE ps.total_quantity_sold > 100  
GROUP BY p.category;
```

Question 4: Using CTEs for Complex Aggregation

Explanation: This query uses a CTE to calculate total product sales based on quantity sold, then filters the products with more than 100 units sold. The main query calculates total sales for those filtered products, grouped by category.

Question 5: Using LEAD() for Analyzing Repeat Orders

Scenario: Calculating the number of days between a customer's consecutive orders.

SOLUTION

```
SELECT o.customer_id,  
       o.order_date,  
       LEAD(o.order_date, 1) OVER (PARTITION BY o.customer_id ORDER  
BY o.order_date) AS next_order_date,  
       DATEDIFF(  
           LEAD(o.order_date, 1) OVER (PARTITION BY o.customer_id  
ORDER BY o.order_date),  
           o.order_date  
       ) AS days_until_next_order  
FROM orders o  
ORDER BY o.customer_id, o.order_date;
```

Explanation: This query uses the `LEAD()` window function to retrieve the order date of a customer's next purchase. It then calculates the number of days between consecutive orders using the `DATEDIFF()` function, partitioning by each customer.



Sravya Madipalli ✓

Was this Helpful?

🔖 Save it

+ Follow Me

↻ Repost and Share it with
your friends